

AI ASSISTED LAB ASSIGNMENT-7.1

Error Debugging with AI.

Name: N.Adithya

Roll No: 2503A52413

Batch: 14

Task-1: Provide a Python snippet with a missing parenthesis in a print statement (e.g., `print("Hello")`). Use AI to detect and fix the syntax error.

Analyze:

Corrected code with proper syntax and AI explanation.

Input:

```
#Missing parentheses in print statement
```

```
print("Hello, World!"
```

```
# solve the error and give the correct code and explanation
```

Explanation:

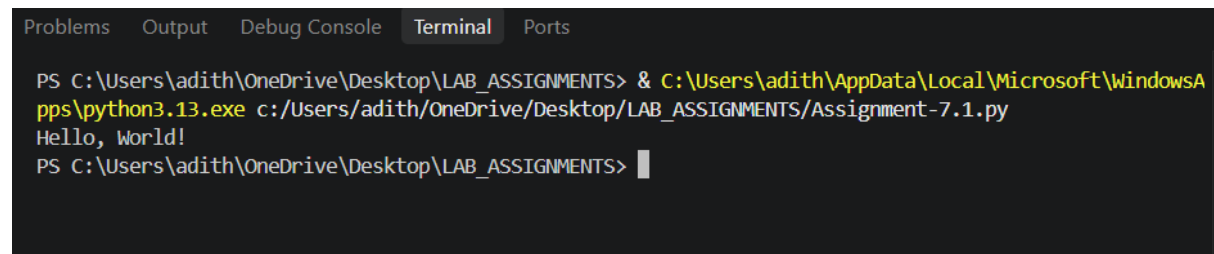
The error in the code is due to a missing closing parenthesis

for the print function. In Python, the print function requires both an opening and a closing parenthesis to properly enclose the string that you want to print.

The corrected code should be:

```
print("Hello, World!")
```

Output:

A screenshot of a Windows PowerShell terminal window. The window has tabs for 'Problems', 'Output', 'Debug Console', 'Terminal', and 'Ports'. The 'Terminal' tab is active. The command prompt shows the user running a command to execute a Python script: `PS C:\Users\adith\OneDrive\Desktop\LAB_ASSIGNMENTS> & C:\Users\adith\AppData\Local\Microsoft\WindowsApps\python3.13.exe c:/Users/adith/OneDrive/Desktop/LAB_ASSIGNMENTS/Assignment-7.1.py`. The output of the script is `Hello, World!`. The prompt then returns to `PS C:\Users\adith\OneDrive\Desktop\LAB_ASSIGNMENTS>` with a cursor.

Task-2: Supply a function where an if-condition

mistakenly uses `=` instead of `==`. Let AI identify and fix the issue.

Input:

```
# Correct the condition that mistakenly used  
# assignment instead of equality.
```

```
def check_number(n):  
    if n == 10:  
        return "Ten"
```

```
else:
```

```
    return "Not Ten"
```

```
# Verification: three successful test cases.
```

```
assert check_number(10) == "Ten"
```

```
assert check_number(9) == "Not Ten"
```

```
assert check_number(11) == "Not Ten"
```

```
print(check_number(10)) # Output: Ten
```

```
print(check_number(9))
```

```
print("All tests passed.")
```

Explanation:

In Python, "=" assigns a value and cannot be used as a condition, causing a `SyntaxError`. The "==" operator compares values, so it correctly checks whether "n" is equal to 10.

Output:

```
PS C:\Users\adith\OneDrive\Desktop\LAB_ASSIGNMENTS> & C:\Users\adith\AppData\Local\Microsoft\WindowsApps\python3.13.exe c:/Users/adith/OneDrive/Desktop/LAB_ASSIGNMENTS/Assignment-7.2.py
Ten
Not Ten
All tests passed.
PS C:\Users\adith\OneDrive\Desktop\LAB_ASSIGNMENTS> █
```

Task-3: Provide code that attempts to open a non-existent file and crashes. Use AI to apply safe error handling.

Input:

```
#generate Provide code that attempts to open a non-existent file and  
# crashes. Use AI to apply safe error handling.
```

```
from pathlib import Path  
from tempfile import TemporaryDirectory  
  
def read_file(filename):  
    """Read a text file and return a user-friendly error instead of  
    crashing."""  
    try:  
        with open(filename, "r", encoding="utf-8") as file:  
            return file.read()  
  
    except FileNotFoundError:  
        return f"Error: The file '{filename}' was not found."  
  
    except (OSError, UnicodeError) as error:  
        return f"Error: Could not read '{filename}': {error}"  
  
def run_tests():
```

```
"""Test an existing file, a missing file, and an invalid path."""
```

```
with TemporaryDirectory() as temporary_directory:
```

```
    existing_file = Path(temporary_directory) / "existing.txt"
```

```
    existing_file.write_text("File contents", encoding="utf-8")
```

```
    print("Existing file:")
```

```
    print(read_file(existing_file))
```

```
    print("Missing file:")
```

```
    print(read_file("nonexistent.txt"))
```

```
    print("Invalid path:")
```

```
    print(read_file("\0invalid-path"))
```

```
if __name__ == "__main__":
```

```
    run_tests()
```

Explanation:

The Python raises an Attribute error. The except block catches it instead of allowing the program to crash.

Output:

```
PS C:\Users\adith\OneDrive\Desktop\LAB_ASSIGNMENTS> & C:\Users\adith\AppData\Local\Microsoft\WindowsApps\python3.13.exe c:/Users/adith/OneDrive/Desktop/LAB_ASSIGNMENTS/Assignment-7.3.py
Existing file:
File contents
Missing file:
Error: The file 'nonexistent.txt' was not found.
Invalid path:
Error: Could not read 'invalid-path': embedded null character
PS C:\Users\adith\OneDrive\Desktop\LAB_ASSIGNMENTS> █
```

Task-4: Give a class where a non-existent method is called (e.g., `obj.undefined_method()`). Use AI to debug and fix.

Input:

```
#Generate Give a class where a non-existent method is
# called (e.g.,
# obj.undefined_method()). Use AI to debug and fix.
```

```
class Car:
```

```
    def start(self):
        return "Car started"
```

```
my_car = Car()
result = my_car.start()
```

```
# Three assert tests confirm that the corrected class and method call work.
```

```
assert isinstance(my_car, Car)
```

```
assert hasattr(my_car, "start")
```

```
assert result == "Car started"
```

```
print("All 3 assert tests passed.")
```

```
print("Output:", result)
```

Explanation:

The original code called "my_car.drive()", but "Car" only defined "start()". Since the intended operation is to start the car, the call is corrected to "my_car.start()" rather than adding an unrelated "drive" method.

Output:

```
PS C:\Users\adith\OneDrive\Desktop\LAB_ASSIGNMENTS> & C:\Users\adith\AppData\Local\Microsoft\WindowsApps\python3.13.exe c:/Users/adith/OneDrive/Desktop/LAB_ASSIGNMENTS/task-7.4.py
All 3 assert tests passed.
Output: Car started
PS C:\Users\adith\OneDrive\Desktop\LAB_ASSIGNMENTS> |
```

Task-5: Provide code that adds an integer and string ("5" + 2) causing a TypeError. Use AI to resolve the bug.

Input:

```
#generate Provide code that adds an integer and string
# ("5" + 2) causing a TypeError. Use AI to resolve
# the bug. # Bug: TypeError due to mixing string and integer

# Solution 1: convert the input to an integer before adding 5.

def add_five(value):
    return int(value) + 5

# Solution 2: convert the integer to a string for concatenation.

def append_five(value):
    return str(value) + "5"

# Validation: three assert test cases.

assert add_five("10") == 15
assert add_five(0) == 5
assert add_five("-2") == 3
```



```
assert append_five("10") == "105"
```

```
assert append_five(0) == "05"
```

```
assert append_five(12) == "125"
```

```
print(add_five("10"))
```

```
print(add_five(7))
```

```
print(append_five("10"))
```

```
print("All tests passed.")
```

Explanation:

Corrected code that runs successfully for multiple inputs.

Type Error caused by mixing strings and integers.

Output:

```
PS C:\Users\adith\OneDrive\Desktop\LAB_ASSIGNMENTS> & C:\Users\adith\AppData\Local\Microsoft\WindowsApps\python3.13.exe c:/Users/adith/OneDrive/Desktop/LAB_ASSIGNMENTS/task-7.5.py
15
12
105
All tests passed.
PS C:\Users\adith\OneDrive\Desktop\LAB_ASSIGNMENTS> █
```